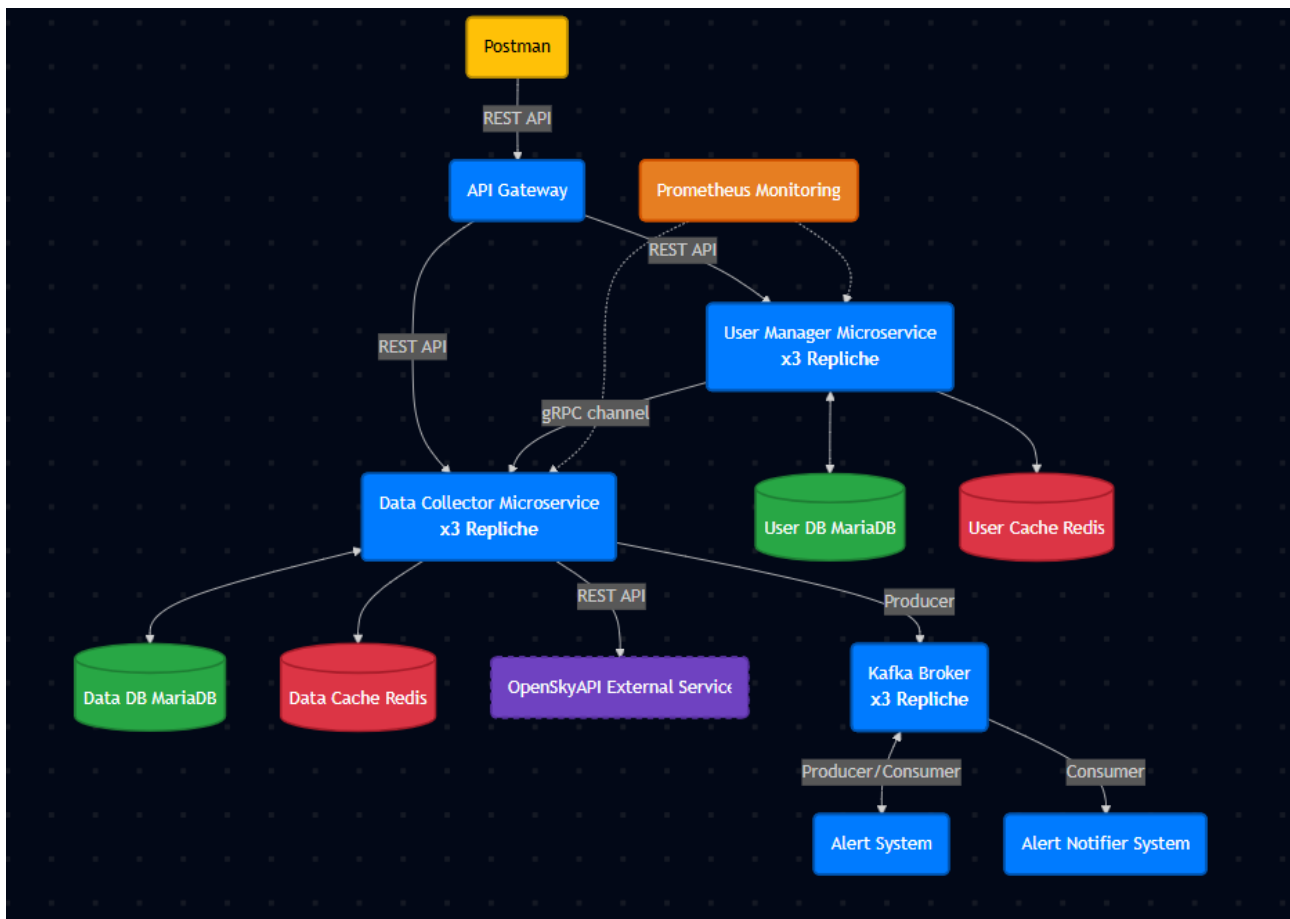


DISTRIBUTED SYSTEM & BIG DATA

HOMEWORK3

DANIELE DI PRIMO – MATTEO SECONDO

1. Panoramica dell'Architettura



L'architettura del sistema è stata concepita come un ecosistema di componenti indipendenti, progettati per operare in sinergia all'interno di un ambiente distribuito. Il design si focalizza sulla modularità e sulla scalabilità orizzontale, dove ogni microservizio è isolato nel proprio container e replicato per garantire la massima continuità operativa.

Questa struttura permette di gestire flussi di dati eterogenei attraverso canali di comunicazione diversificati: dalle interfacce REST dedicate al client, ai protocolli ad alte prestazioni (**gRPC**) per lo scambio di informazioni interno tra i servizi. L'intera infrastruttura è stata inoltre integrata con sistemi per la telemetria e la gestione asincrona degli eventi, assicurando che il sistema sia non solo resiliente, ma anche osservabile.

Il sistema è strutturato in:

- A. Layer di Ingresso e Orchestrazione.** L'accesso dall'esterno verso i servizi è mediato da un **API Gateway**. Questo componente agisce come unico punto di ingresso (*Single Entry Point*) per il client,

gestendo il routing delle richieste verso i servizi di backend appropriati e nascondendo la complessità della topologia interna.

B. Layer dei Core Microservices. Questo livello gestisce la logica di business ed è composto da:

- **User Manager:** Responsabile della gestione delle identità. Per garantire alta disponibilità e gestire carichi elevati, questo microservizio è stato scalato orizzontalmente con una configurazione a 3 repliche attive.
- **Data Collector:** Responsabile dell'interazione con OpenSky Network per il reperimento dei dati di volo. Oltre a rispondere alle richieste dirette, ora agisce come *Producer*, immettendo i dati elaborati nel sistema di messaging. Anche questo microservizio ha 3 repliche.
- **Comunicazione:** L'API Gateway comunica con i due servizi tramite REST, mentre la comunicazione diretta tra User Manager e Data Collector continua a beneficiare delle performance di **gRPC** (in verso opposto rispetto alla versione homework2).

C. Layer Event-Driven (Asincrono) Per l'elaborazione dei dati e la gestione degli allarmi in tempo reale, è stato introdotto un sistema di messaggi basato su **Apache Kafka**. Questo abilita un flusso di dati asincrono:

- **Kafka Broker:** Riceve i flussi di dati dal Data Collector e li distribuisce ai consumatori interessati.
- **Alert System:** Un microservizio che si sottoscrive ai topic Kafka per analizzare i dati di volo in tempo reale e determinare, in base a regole di business, se attivare un allarme.
- **Alert Notifier System:** Un componente dedicato esclusivamente alla gestione delle notifiche tramite email verso gli utenti, attivato dagli eventi presenti su Kafka.
- **Kafka UI:** Espone un'interfaccia che permette la visualizzazione dei topic utilizzati, dei messaggi in arrivo, e dei consumer che sono sottoscritti ad essi.

L'adozione di Kafka e la separazione dei sistemi di Alert garantisce che l'elaborazione pesante e l'invio delle notifiche non impattino sulle prestazioni dei servizi principali rivolti all'utente.

D. Sistema di monitoraggio. Per il collezionamento di metriche riguardanti sia lo stato dei microservizi, sia statistiche a livello applicativo, è stato integrato nell'architettura un servizio centralizzato basato su **Prometheus**. Tale componente opera attraverso un modello di tipo *pull*, effettuando lo *scraping* periodico di endpoint HTTP dedicati esposti da ciascun microservizio. L'adozione di questa soluzione permette di storicizzare in un database informazioni eterogenee: dalle **metriche infrastrutturali** (come la disponibilità dei nodi e l'utilizzo delle risorse) alle **metriche di business** (come i tempi di risposta delle API e i contatori delle operazioni utente). Questo garantisce un'osservabilità completa sulle performance e sull'affidabilità del sistema distribuito.

2. Descrizione dei Moduli

A. API Gateway

Componente che funge da *Single Entry Point* per l'intera architettura, mascherando la complessità della rete interna. Il nodo, istanziato in duplice copia (due repliche), è strutturato su due moduli principali provenienti dall'immagine **openresty (NGINX + LUA)**:

- **Nginx Web Server:** Il componente **Nginx** ricopre il ruolo fondamentale di Reverse Proxy e gestore del traffico. La sua configurazione primaria prevede l'intercettazione di tutte le richieste in ingresso, imponendo un upgrade di protocollo tramite il reindirizzamento automatico da HTTP a HTTPS per garantire la cifratura delle comunicazioni. Esso applica una logica di **instradamento basata sui percorsi (path-based routing)**. Analizzando il prefisso dell'URI richiesto dal client, il server smista il traffico verso lo specifico microservizio di competenza: le chiamate destinate a /users vengono dirette al microservizio User Manager, quelle verso /flights sono instradate al Data Collector, mentre il traffico su /auth viene delegato al componente User Manager per le operazioni di autenticazione e registrazione.
- **Script Lua:** Il nodo prevede l'impiego di uno **script Lua** per gestire la logica avanzata di validazione e controllo. Questo modulo, infatti, non si limita alla validazione crittografica standard del JWT, ma introduce un ulteriore livello di sicurezza contestuale. Prima di inoltrare qualsiasi richiesta ai microservizi, lo script LUA controlla lo stato dell'utente attraverso una **BlackList** per accertarsi che l'account sia attivo. Un utente viene inserito nella suddetta lista se procede alla cancellazione del profilo e, con un JWT ancora valido, richiede servizi appartenenti a root private. Il sistema, allora, applica una **politica di revoca immediata**: l'accesso viene inibito istantaneamente prevenendo l'utilizzo di token che, seppur formalmente validi, appartengono a utenze non più esistenti.

B. User Manager Service

Microservizio core responsabile dell'intero ciclo di vita delle identità utente. Il servizio è progettato per garantire l'**Alta Disponibilità (HA)** tramite un deployment su repliche multiple, minimizzando i single-point-of-failure.

Interfacce Esposte

- **POST /auth/register:** Registrazione nuovo utente.
 - *Payload:* Email (PK), Password, Nome, Cognome.
- **POST /auth/login:** Autenticazione utente.
 - *Input:* Credenziali (Email, Password).
 - *Output:* Rilascio del **JWT** per le chiamate successive.
- **DELETE /users/delete:** Cancellazione account e relative informazioni.
 - *Input:* Email utente.

Data Layer

- **Persistenza (MariaDB):** Database relazionale contenente la "Single Source of Truth" per i dati anagrafici. La chiave primaria è l'email.
- **Caching & Performance (Redis):** Istanza in-memory utilizzata per:
 1. Cache delle letture frequenti (riduzione carico DB).
 2. Gestione dell'idempotenza e per evitare l'esecuzione multipla di funzioni già eseguite.

C. Data Collector Service

Il Data Collector è il microservizio ibrido responsabile dell'acquisizione e normalizzazione dei dati di traffico aereo. Opera con una duplice modalità: **reattiva** (risposta sincrona alle chiamate REST) e **proattiva** (esecuzione di task asincroni programmati). Agisce inoltre come **Kafka Producer**, alimentando la pipeline di allertamento.

Interfacce Esposte (Gateway Prefix: /flights)

- **POST /airport-of-interest/add:** Configurazione monitoraggio aeroporto.
 - *Funzione:* Sottoscrive un aeroporto specifico agli aggiornamenti periodici.
 - *Alerting:* Permette l'impostazione opzionale di soglie (high_value, low_value). Se i dati acquisiti violano questi parametri, il servizio pubblica un evento sul topic Kafka per innescare la notifica email.
- **GET /get-flights/latest:** Recupero dati real-time.
 - *Output:* Ultimo movimento (arrivo/partenza) registrato per l'aeroporto richiesto.
- **GET /airport-of-interest/average:** Analisi statistica.
 - *Output:* Calcolo della media e del totale dei voli su una finestra temporale (Time Window) specificata.

Oltre a esporre un'interfaccia REST per la consultazione puntuale e statistica dei voli, il servizio implementa una logica di ingestion asincrona. Attraverso l'utilizzo di Kubernetes CronJobs, il sistema esegue ciclicamente (ogni 8 ore) il fetch dei dati dalla sorgente esterna *OpenSky Network*, focalizzandosi esclusivamente sugli aeroporti di interesse mappati dagli utenti.

Data Layer

- **Persistenza (MariaDB):** Database relazionale strutturato in due tabelle: AirportsOfInterests, Flights. La prima contiene gli aeroporti di interesse (ICAO) associati agli utenti, la seconda contiene le informazioni sui voli il cui ICAO è presente in AirportsOfInterests.
- **Caching & Performance (Redis):** Istanza in-memory utilizzata per:
 1. Cache delle letture frequenti (riduzione carico DB).
 2. Gestione dell'idempotenza e per evitare l'esecuzione multipla di funzioni già eseguite.

D. Kafka

Per garantire un disaccoppiamento efficace tra il layer di acquisizione dati e la logica di notifica, l'architettura implementa un pattern di comunicazione asincrona basato su **Confluent Kafka**. Questa scelta progettuale risulta strategica per gestire eventuali picchi di traffico e per isolare i processi di elaborazione (*Alert System*) e distribuzione (*Notifier*) dal ciclo di vita del *Data Collector*.

Infrastruttura e Modalità Kraft. Al fine di assicurare l'Alta Disponibilità (HA) e la persistenza affidabile dei dati, il cluster Kafka è stato distribuito all'interno dell'ambiente Kubernetes mediante uno **StatefulSet** composto da tre repliche. L'architettura adotta la moderna modalità **KRaft (Kafka Raft Metadata mode)**, eliminando la dipendenza storica da ZooKeeper. In questa configurazione, il consenso distribuito e l'elezione del leader sono gestiti internamente ai nodi tramite un meccanismo di quorum, con i nodi configurati per operare simultaneamente sia come *broker* che come *controller*. La comunicazione intra-cluster e la corretta gestione degli *Advertised Listeners* sono garantite da un **Service Headless**, che assegna identità di rete stabili e nomi DNS predicibili a ciascun pod.

Pipeline di Elaborazione Messaggi. Il flusso dei dati è orchestrato attraverso una pipeline a due stadi che coinvolge due topic distinti e tre microservizi coordinati:

1. **Ingestione Dati (Topic: to-alert-system).** Il *Data Collector Service* agisce come **Producer primario**. Al termine della fase di fetch dati da OpenSky, per ogni aeroporto monitorato, il servizio costruisce un payload JSON contenente: il numero attuale di voli rilevati, le soglie di allarme configurate dall'utente (*high_value* e *low_value*) e l'identificativo dell'utente (*email*). Questo messaggio viene pubblicato sul topic *to-alert-system* senza l'applicazione di alcuna logica di controllo: il *Data Collector* si limita a fotografare e trasmettere lo "stato attuale" del sistema.
2. **Elaborazione e Filtraggio (Alert System).** L'*Alert System* opera come **Consumer** sul topic *to-alert-system*. La sua funzione è puramente logica: per ogni messaggio ricevuto, confronta il numero di voli rilevati con le soglie incluse nel payload. Se viene rilevata una violazione delle regole di business (ovvero se *voli_attuali* > *high_value* per traffico eccessivo, o *voli_attuali* < *low_value* per traffico scarso), il sistema determina la necessità di inviare un avviso. In tal caso, l'*Alert System* assume il ruolo di **Producer**, pubblicando un nuovo evento sul topic *to-notify*. I messaggi che non violano le soglie vengono scartati, filtrando efficacemente il "rumore" a monte del sistema di notifica.
3. **Distribuzione Notifiche (Topic: to-notify).** L'*Alert Notifier System* rappresenta il consumatore finale della pipeline. Sottoscrivendo il topic *to-notify*, riceve esclusivamente eventi "confermati" che richiedono un'azione concreta. Il servizio preleva il messaggio e gestisce l'invio effettivo dell'email all'utente finale tramite protocollo SMTP.

Strategia di Consumo e Latenza. Attualmente, i Consumer sono configurati con un parametro *BATCH_SIZE* = 1. Questa scelta privilegia la bassa latenza (*Real-Time*) a discapito del throughput massimo: ogni messaggio viene prelevato, processato ed il relativo offset viene committato singolarmente. Ciò garantisce che l'utente riceva l'alert nell'istante esatto in cui il dato viene analizzato, minimizzando i tempi di attesa.

Osservabilità (Kafka UI). A supporto della gestione operativa, l'architettura integra **Kafka UI**, un'interfaccia web open-source per l'ispezione visiva del cluster. L'adozione di questo strumento si è rivelata essenziale durante le fasi di sviluppo e testing per monitorare in tempo reale la creazione e il popolamento dei topic critici (*to-alert-system* e *to-notify*). La dashboard ha permesso di analizzare la struttura dei payload JSON scambiati tra i microservizi, validando la correttezza dei dati senza ricorrere a comandi da terminale, e ha

offerto visibilità immediata sullo stato dei **Consumer Group**, facilitando l'identificazione di eventuali ritardi (lag) nel processamento.

Automazione del Provisioning (Job di Inizializzazione) A completamento dell'architettura, la gestione del ciclo di vita delle risorse logiche Kafka è affidata a un **Job Kubernetes** dedicato, denominato kafka-init. Questo componente *batch* ha il compito cruciale di automatizzare la configurazione iniziale del cluster, eliminando la necessità di interventi manuali e garantendo la riproducibilità dell'ambiente.

Il Job implementa una logica di **sincronizzazione robusta**: all'avvio, esegue un ciclo di polling verso i server di bootstrap (raggiungibili tramite gli indirizzi DNS stabili forniti dal servizio headless), mettendo in pausa l'esecuzione finché il cluster non risulta pienamente operativo e raggiungibile.

Una volta stabilita la connessione, lo script procede alla creazione idempotente dei topic core (to-alert-system e to-notify) utilizzando il flag `--if-not-exists`. Coerentemente con i requisiti di Alta Disponibilità, i topic vengono configurati con un **Replication Factor pari a 3** (per garantire una copia del dato su ogni broker) e partizionati per consentire un eventuale scaling orizzontale dei consumatori.

E. Prometheus

Per garantire l'osservabilità del sistema distribuito, è stata implementata un'infrastruttura di monitoraggio basata su Prometheus, orchestrata direttamente all'interno del cluster Kubernetes. La configurazione del server di monitoraggio è stata definita tramite una **ConfigMap** integrata nel manifesto di deployment, abbandonando l'approccio a target statici in favore della **Kubernetes Service Discovery**. In questo scenario dinamico, Prometheus è istruito per scansionare automaticamente l'intero cluster e identificare i target di scraping basandosi sulle *Annotations* presenti nei manifesti dei singoli microservizi: nello specifico, vengono interrogati i Pod che presentano l'annotazione ***prometheus.io/scrape: "true"*** sulla porta definita in ***prometheus.io/port***. Questo approccio garantisce che ogni nuova replica o servizio rilasciato venga immediatamente monitorato senza necessità di riconfigurare il server centrale.

Dal punto di vista applicativo, l'integrazione è stata realizzata mediante la libreria ***prometheus-client*** per Python. Per far fronte alla natura effimera dei Pod in Kubernetes, ogni metrica è arricchita semanticamente con etichette (***labels***) che identificano il servizio (***service_name***) e l'istanza specifica (***pod_name***, ***node_name***).

Le metriche esposte dai microservizi si dividono in due macrocategorie. Le metriche di tipo **Counter**, utilizzate per valori cumulativi come il conteggio delle richieste totali o degli errori, includono:

- **HTTP_REQUESTS_TOTAL**: Contatore delle richieste http dirette ai microservizi. Presente sia nello User-Manager che nel Data-Collector.
- **ERROR_COUNT_OPENSky**: Contatore dei fallimenti della funzione *add-airports-of-interests* a causa di chiamata esterna ad OpenSky fallita.
- **ERROR_COUNT**: Contatore dei fallimenti della funzione *add-airports-of-interests* a causa di errori generici.
- **REQUESTS_COUNT**: Contatore delle richieste http dirette alla funzione *add-airports-of-interests*.
- **REGISTER_REQUESTS_COUNT**: Contatore delle richieste http dirette alla funzione di registrazione presente nello User-Manager.
- **LOGIN_REQUESTS_COUNT**: Contatore delle richieste http dirette alla funzione di login presente nello User-Manager.

Le metriche di tipo **Gauge**, utilizzate invece per valori oscillanti come i tempi di risposta istantanei o la saturazione delle risorse, comprendono:

- **LATEST_RESPONSE_TIME**: Calcola il tempo di risposta dell'ultima chiamata alla funzione `add-airports-of-interests`.
- **LATEST_RESPONSE_TIME_BACKGROUND_TASK**: Calcola il tempo di risposta della funzione `background_task`, responsabile dell'inserimento in DB dei voli d'interesse aggiunti dall'utente.

4. Scelte Progettuali e Tecnologie

A. Orchestrazione e Infrastruttura: Kubernetes su Kind

La gestione del ciclo di vita dei microservizi è affidata a **Kubernetes**, standard de facto per l'orchestrazione di container. Per garantire un ambiente di sviluppo locale che fosse fedele alla topologia di una produzione reale, ma al contempo leggero e facilmente riproducibile, è stato adottato lo strumento **Kind (Kubernetes in Docker)**.

Topologia del Cluster. A differenza di configurazioni locali standard (come Minikube a nodo singolo), il cluster è stato definito in modalità **Multi-Node** tramite manifest dichiarativi. La configurazione prevede:

- **1 Nodo Control-Plane (Master)**: Responsabile della gestione dello stato del cluster. Questo nodo è stato configurato con l'etichetta `ingress-ready=true` e con un mapping diretto delle porte (`hostPort 80` e `443`) verso il container. Ciò permette al controller `Nginx Ingress` di ricevere traffico HTTP/HTTPS direttamente da `localhost`, emulando un Load Balancer reale.
- **3 Nodi Worker**: Tre nodi di elaborazione dedicati all'esecuzione dei pod applicativi. Questa scelta non è casuale ma necessaria per supportare i requisiti di **Alta Disponibilità (HA)** del progetto: la presenza di tre worker fisici (simulati su container Docker distinti) permette infatti il corretto scheduling delle **3 repliche** di Kafka, User Manager e Data Collector, garantendo la resilienza del sistema anche in caso di caduta di un nodo.

Giustificazione della Scelta. L'adozione di Kubernetes tramite Kind offre vantaggi decisivi:

1. **Isolamento**: Ogni nodo è un container Docker, garantendo un ambiente pulito che non interferisce con il sistema operativo host.
2. **Infrastructure as Code (IaC)**: L'intera infrastruttura è definita nel file `kind-config.yaml`. Questo rende il setup deterministico: chiunque possieda il file può ricreare l'esatta infrastruttura con un singolo comando.
3. **Parità con la Produzione**: Permette di testare scenari complessi di distribuzione del carico prima del deploy effettivo.

I servizi sono sviluppati in **Flask (Python)**, eseguendo parallelamente server REST e gRPC. La persistenza è affidata a **MariaDB** con ORM **SQLAlchemy** per sicurezza e pooling delle connessioni.

Strategia di Caching e Politica At-Most-Once

Elemento cardine dell'architettura è l'implementazione della semantica **At-Most-Once** tramite **Redis**. La scelta di questa tecnologia deriva dalla sua operatività in-memory (RAM), che assicura velocità estreme e bassa latenza. Inoltre, la funzionalità nativa di *Time-To-Live (TTL)* permette una gestione automatica e precisa della scadenza dei dati, mentre la sua natura di server centralizzato garantisce una cache condivisa e scalabile, ideale per l'ecosistema a microservizi.

L'utilizzo della cache è stato declinato in modo duale per rispondere a esigenze specifiche dei due moduli:

- **User Manager (Garanzia di Idempotenza):** In questo contesto, Redis è essenziale per la consistenza dei dati. Il sistema implementa un meccanismo di controllo basato sugli header HTTP X-REQUEST-ID (identificativo della richiesta) e X-CLIENT-ID (identificativo del client). Prima di eseguire operazioni critiche come la registrazione, il servizio verifica la presenza di questa coppia di chiavi in cache. Questo permette di intercettare eventuali *retry* di rete automatici, prevenendo l'esecuzione duplicata di codice sul database e garantendo che ogni operazione venga processata al massimo una volta.
- **Data Collector (Ottimizzazione delle Performance):** In questo modulo, l'uso di Redis è orientato all'efficienza computazionale e alla riduzione del carico, seppur il meccanismo di controllo basato sugli header HTTP X-REQUEST-ID e X-CLIENT-ID è uguale a quello implementato nello User Manager. La cache viene utilizzata per memorizzare i risultati di query onerose in termini di risorse dati e dati frequentemente richiesti (*hot data*). Questo approccio minimizza le letture sul database relazionale e riduce la latenza nelle risposte all'utente finale.

C. Circuit Breaker

L'architettura del sistema integra il pattern **Circuit Breaker** con l'obiettivo di garantire la resilienza e prevenire i cosiddetti *cascading failures* (fallimenti a cascata). Nello specifico, questo modulo agisce da intermediario per tutte le interazioni con il servizio esterno OpenSky.

Il meccanismo opera monitorando il tasso di errore delle chiamate: qualora il servizio OpenSky risulti irraggiungibile o instabile oltre una certa soglia, il circuito commuta nello stato **OPEN**. In questa fase, le richieste successive vengono bloccate preventivamente, restituendo un errore immediato senza saturare le risorse di rete.

Trascorso un periodo di *cooldown* (o *recovery*), il sistema transita nello stato **HALF-OPEN**, permettendo l'inoltro di un numero limitato di richieste di prova. Se queste hanno esito positivo, il circuito viene ripristinato (**CLOSED**), tornando alla normale operatività; in caso contrario, il circuito si riapre, ricominciando il ciclo di protezione.

D. Comunicazione Inter-Service: gRPC

Sebbene la comunicazione verso i client esterni avvenga tramite API REST standard, per le interazioni critiche tra microservizi backend si è scelto di adottare il protocollo **gRPC**. Questa decisione è dettata dalla necessità di ottimizzare le performance per le chiamate sincrone dirette (*service-to-service*).

Un'applicazione chiave di questa tecnologia riguarda la gestione del **ciclo di vita dell'utenza**. Nel momento in cui un utente richiede la cancellazione del proprio account, il sistema deve garantire non solo la rimozione delle credenziali, ma anche la pulizia di tutti i dati accessori (come gli aeroporti di interesse) residenti in altri domini logici. Utilizzando gRPC, il microservizio *User-Manager* può invocare una procedura remota sul *Data-Collector* con latenza quasi nulla e tipizzazione forte. Questo meccanismo assicura una **consistenza dei dati "eventuale ma rapida"**, prevenendo l'accumulo di dati orfani nel database dei voli senza l'overhead tipico delle chiamate HTTP tradizionali.

E. SSL

Nell'ottica di adottare le best practice di sicurezza sin dalle prime fasi di sviluppo, l'architettura è stata progettata per garantire la cifratura del traffico in transito tramite protocollo HTTPS. Per l'ambiente di esecuzione locale, non disponendo di un dominio pubblico validato da un'autorità di certificazione (CA), si è optato per la generazione manuale di certificati SSL autofirmati (Self-Signed Certificates).

Questa configurazione è applicata a livello dell'API Gateway, che agisce come terminatore SSL: il Gateway espone la porta sicura 443 verso l'esterno, gestendo l'handshake crittografico con il client (Postman o Browser) e decifrando le richieste prima di inoltrarle ai microservizi interni su rete privata. Sebbene questa soluzione comporti la visualizzazione di avvisi di sicurezza nei client standard (poiché l'autorità emittente non è riconosciuta pubblicamente), essa permette di simulare fedelmente uno scenario di produzione, validando le configurazioni dei web server e garantendo che nessun dato sensibile, come le credenziali utente o i token, viaggi in chiaro sulla rete. In una futura fase di deploy in produzione, questa infrastruttura è predisposta per la sostituzione trasparente dei certificati manuali con quelli validi rilasciati da autorità.

F. Gestione dell'Autenticazione e Sicurezza tramite JWT

L'architettura di sicurezza del sistema si basa sullo standard JSON Web Token (JWT), implementato attraverso un pattern asimmetrico (RS256) che coinvolge il microservizio di gestione utenti per l'emissione e il gateway Nginx per la validazione.

Motivazioni e Vantaggi della Scelta

L'adozione di JWT risponde all'esigenza di creare un sistema di autenticazione distribuito e performante. Il vantaggio principale risiede nella **natura "Stateless" e autocontenuta** del token: il JWT trasporta al suo interno tutte le informazioni necessarie (payload) per identificare l'utente (come email e client_id). Questo elimina la necessità per i microservizi di interrogare costantemente il database centrale (tramite la precedente comunicazione gRPC per il controllo dell'esistenza della mail sullo User Manager) per validare la sessione ad ogni singola richiesta HTTP, riducendo drasticamente la latenza di rete e il carico.

Inoltre, l'utilizzo dell'algoritmo di firma asimmetrica **RS256** garantisce una rigorosa **separazione delle responsabilità**:

- La **Chiave Privata** risiede esclusivamente nel microservizio *User Manager* (l'unico autorizzato a creare/firmare token).
- La **Chiave Pubblica** è distribuita al Gateway Nginx (che può solo verificare la validità dei token, ma non crearne di falsi). Questo approccio eleva il livello di sicurezza: anche in caso di compromissione del Gateway, un attaccante non potrebbe mai generare credenziali valide per impersonare altri utenti.

Il Flusso di Autenticazione

Il processo si articola in due fasi distinte, visibili nel codice Python (emissione) e Lua (validazione):

- **Fase di Emissione (Login):** Quando l'utente invia le proprie credenziali all'endpoint `/auth/login`, il microservizio verifica la password e, in caso di successo, genera un JWT firmato con la Chiave

Privata. All'interno del token vengono incapsulati i *claims* fondamentali: l'identificativo utente (sub), un identificativo univoco di sessione (client_id) e la data di scadenza (exp).

- **Fase di Intercettazione e Inoltro:** Per tutte le chiamate successive, il client invia il token nell'header Authorization. Qui interviene lo script Lua integrato in Nginx. Prima ancora che la richiesta raggiunga i microservizi interni (come il Data Collector), il Gateway intercetta il pacchetto ed esegue la verifica crittografica della firma usando la Chiave Pubblica caricata in memoria RAM.
- **Header Injection:** Se la firma è valida, lo script estrae le informazioni dal payload del token (es. l'email) e le inietta nella richiesta sotto forma di header HTTP standard (X-User-Email). In questo modo, i microservizi a valle ricevono l'identità dell'utente già validata, rimanendo completamente agnostici rispetto alla complessità della crittografia JWT.

Revoca del Token e Gestione Utenti Cancellati

Una delle criticità intrinseche dei JWT è l'impossibilità di revocarli prima della loro scadenza naturale. Per mitigare questo rischio e gestire casi critici come l'eliminazione dell'account o il furto di credenziali, l'architettura implementa un controllo ibrido (Stateful) tramite Redis.

Nello script Lua, dopo la verifica formale della firma, viene eseguito un controllo puntuale:

1. Viene estratto il client_id dal payload del token.
2. Viene effettuata una query rapidissima in-memory verso l'istanza **Redis Cache** per verificare se tale ID è presente in una **Blacklist**.

Se un utente decide di eliminare il proprio account, il sistema non si limita a cancellare il record dal database, ma inserisce il corrispondente client_id nella blacklist di Redis. Di conseguenza, anche se l'utente possiede ancora un token tecnicamente valido (non scaduto), il Gateway bloccherà immediatamente ogni ulteriore richiesta restituendo un errore 401, garantendo così una revoca dell'accesso istantanea e sicura.

4. Topologia di Rete e Service Discovery

In ambiente Kubernetes, la natura effimera dei Pod (che possono essere distrutti e ricreati con IP differenti in qualsiasi momento) rende impossibile basare la comunicazione su indirizzi IP diretti e volatili. Per risolvere questa problematica, l'architettura si affida all'astrazione dei **Service Kubernetes**, che garantiscono un punto di accesso stabile e invariabile per la comunicazione tra microservizi.

Il meccanismo di comunicazione si basa sul **Service Discovery** interno gestito da CoreDNS. Quando un microservizio necessita di contattarne un altro, non interroga un IP, ma risolve il nome di dominio del servizio (es. user-service.default.svc.cluster.local).

Sulla base delle esigenze specifiche dei componenti, sono state adottate due strategie di networking distinte:

1. **Comunicazione Standard (ClusterIP):** Per la maggior parte del traffico HTTP/REST (es. tra Gateway e microservizi), viene utilizzato il pattern **ClusterIP**. Kubernetes assegna al Service un indirizzo IP virtuale (VIP) stabile. Le richieste inviate a questo VIP vengono intercettate da kube-proxy, che applica una logica di **Load Balancing (Round Robin)** a livello 4 (trasporto), distribuendo il traffico

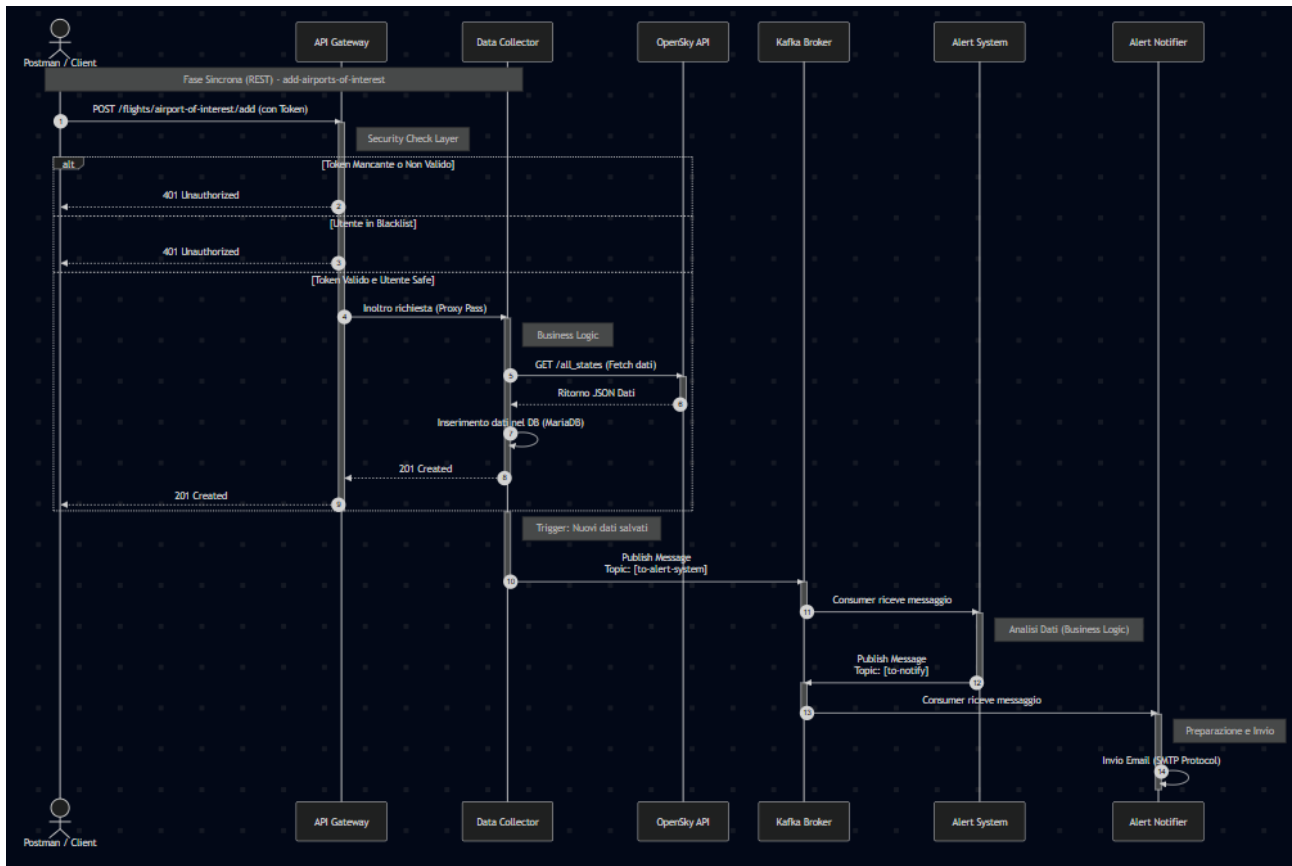
equamente tra tutte le repliche disponibili (Pod) del microservizio di destinazione. Questo approccio è ideale per servizi stateless dove ogni replica è equivalente.

2. **Comunicazione Diretta (Headless Service):** Per componenti complessi come **Kafka** e per comunicazioni ad alte prestazioni via **gRPC**, è stato adottato il pattern **Headless**. In questa configurazione, il Service non possiede un IP virtuale unico. Al contrario, la risoluzione DNS restituisce direttamente la lista degli indirizzi IP di tutti i Pod sottostanti.
 - **Per Kafka (Stateful):** È fondamentale perché i broker hanno identità distinte e i client devono sapere esattamente quale specifico broker contattare (es. il leader di una partizione).
 - **Per gRPC:** Permette di implementare il *client-side load balancing*. Poiché gRPC mantiene connessioni persistenti, un load balancer tradizionale (ClusterIP) rischierebbe di inviare tutte le richieste alla stessa replica; con un servizio Headless, il client conosce tutti i Pod disponibili e può distribuire le chiamate in modo intelligente.
-

5. Diagramma delle interazioni

Viene di seguito illustrato il diagramma di sequenza relativo allo scenario "Add Airports of Interest". Tale casistica è stata selezionata poiché rappresenta il flusso più articolato dell'architettura. Il processo ha inizio con una **fase di validazione** gestita dall'API Gateway, che agisce da filtro di sicurezza verificando la validità del token JWT e l'eventuale presenza dell'utente nella *blacklist*; solo le richieste legittime vengono inoltrate al *Data Collector*. Quest'ultimo esegue le operazioni "core" in modalità sincrona: recupera i dati da *OpenSkyAPI*, li persiste nel database MariaDB e restituisce immediatamente al client un codice 201 Created.

Solo a questo punto si attiva il processo asincrono: il *Data Collector* pubblica un evento su Kafka, delegando l'elaborazione delle notifiche all'*Alert System* e al successivo *Notifier*, garantendo così che l'invio delle email non influisca sulla latenza della risposta all'utente.



6. Istruzioni per l'esecuzione del codice

Vedere README.md presente nella repository GitHub.